LABORATORY EQUIPMENT MANAGEMENT

1. Problem Statement

A college laboratory contains different types of equipment such as microscopes, computers, measuring instruments, and other practical devices. Managing this equipment manually through registers or spreadsheets can make it difficult to keep track of equipment details and current availability. Laboratory staff may face problems in identifying which equipment is available, which has been issued, and which has been returned.

To solve this problem, a Laboratory Equipment Management System is developed using Solidity. The system stores important details such as equipment ID, equipment name, equipment type, and availability status on the blockchain through a smart contract. It provides a menu-driven approach through a single menu function, where the user supplies the required values as function inputs to add equipment, issue equipment, return equipment, or view equipment details.

The smart contract checks the availability of an equipment item before issuing it. If an equipment item has already been issued, the contract prevents it from being issued again. When the equipment is returned, its status is changed back to available. The contract also handles invalid operations such as entering an incorrect equipment ID, adding a duplicate ID, selecting an invalid menu option, or entering empty equipment details.

Exception handling is implemented using Solidity validation mechanisms such as require and revert. These mechanisms stop an invalid transaction and display an appropriate error message instead of allowing an incorrect state change. Thus, the proposed smart contract provides a simple and organized method for maintaining laboratory equipment records and tracking their availability on a blockchain.

2. Solidity Program

```
 // SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract LaboratoryEquipmentManagement {

    struct Equipment {
        uint id;
        string name;
        string equipmentType;
        bool available;
    }

    mapping(uint => Equipment) public equipment;

    // Store equipment details
    function addEquipment(
        uint _id,
        string memory _name,
        string memory _type
    ) public {
        require(_id > 0, "Invalid ID");
        require(
            bytes(equipment[_id].name).length == 0,
            "Equipment already exists"
        );

        equipment[_id] = Equipment(
            _id,
            _name,
            _type,
            true
        );
    }
```

```
// Issue equipment
function issueEquipment(uint _id) public {

  require(

      bytes(equipment[_id].name).length > 0,
      "Equipment not found"
   );
   require(
      equipment[_id].available,
      "Equipment not available"
   );

   equipment[_id].available = false;
}

// Return equipment
function returnEquipment(uint _id) public {
   require(
      bytes(equipment[_id].name).length > 0,
      "Equipment not found"
   );
   require(
      !equipment[_id].available,
      "Already available"
   );

   equipment[_id].available = true;
}

// Display equipment details
function getEquipment(uint _id)
   public
   view
   returns (uint, string memory, string memory, bool)
{
   require(
      bytes(equipment[_id].name).length > 0,
      "Equipment not found"
   );
```

```
    Equipment memory e = equipment[_id];
        return (
            e.id,
            e.name,
            e.equipmentType,
            e.available
        );
    }
}
```

3. Brief Explanation

Smart Contract and Mapping

The LaboratoryEquipmentManagement smart contract manages laboratory equipment. A Solidity mapping stores each equipment record using the equipment ID as the key. The Equipment structure contains the ID, name, type, availability status, and an existence flag.

Adding Equipment

The addEquipment() function stores the equipment ID, name, type, and availability status. Every newly added equipment is initially marked as available. The require statements validate the ID, name, type, and duplicate ID condition before storing the record.

Issuing Equipment

The issueEquipment() function first checks whether the equipment exists and whether it is available. If available, its status is changed to false. If it has already been issued, the transaction is rejected using require.

Returning Equipment

The returnEquipment() function changes the status of issued equipment back to available. It checks whether the equipment exists and whether it is currently unavailable before updating the record.

Retrieving Equipment

The getEquipment() function returns the current ID, name, type, and availability status of the selected equipment. It is declared as a view function because it only reads blockchain data.

Menu-Driven Operation

The menu() function provides a single menu-driven entry point. Choice 1 adds equipment, choice 2 issues equipment, choice 3 returns equipment, and choice 4 emits the equipment details. In Remix, the user provides these values through the function input fields because Solidity does not use console-style input() statements.

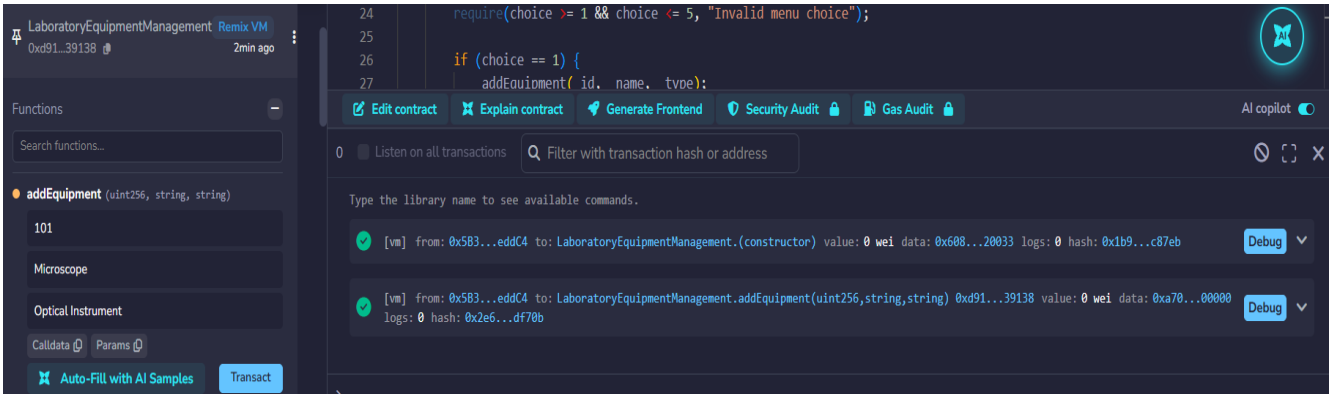
Exception Handling

Solidity handles invalid operations using require and revert. require checks conditions such as valid IDs, existing equipment, availability, duplicate entries, and valid menu choices. If a condition fails, the transaction is reverted and the specified error message is displayed.

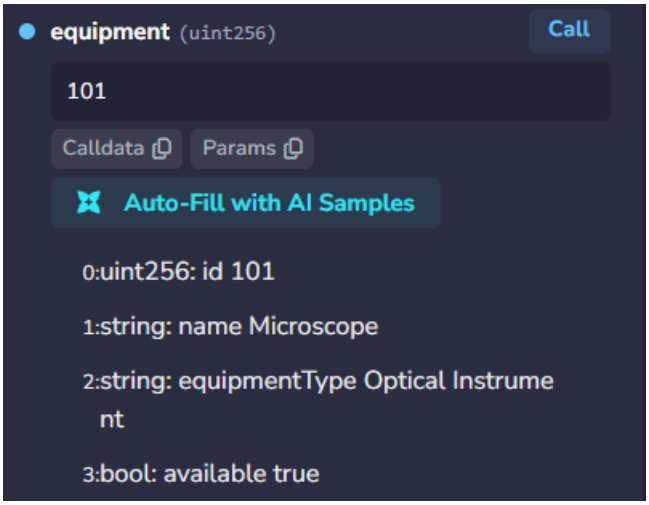
4. Output Screenshots

The following outputs demonstrate the working of the Solidity smart contract in Remix IDE. The user enters values through the function input fields, and successful transactions or reverted transactions show the corresponding result.

Output 1 – Equipment Added



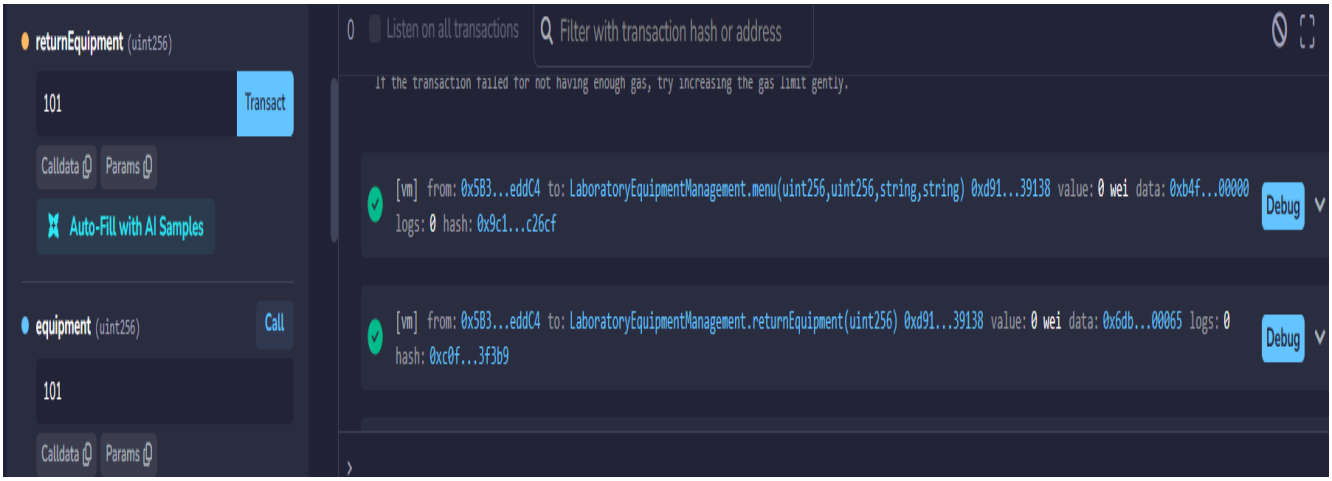
Output 2 – Equipment Details



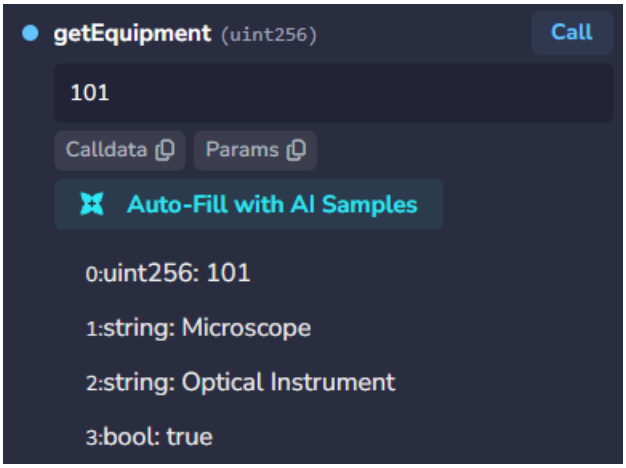
Output 3 – Equipment Issued



Output 4 – Return Equipment



Output 5 – Fetch Equipment



5. Conclusion

The Laboratory Equipment Management System was successfully implemented as a Solidity smart contract to simplify the management of laboratory equipment. The contract maintains equipment details such as ID, name, type, and availability status in a structured and transparent manner on the blockchain.

The smart contract allows the user to add equipment, issue available equipment, return issued equipment, and view equipment details. It checks the availability status before issuing equipment, thereby preventing an already-issued equipment item from being issued again. The return operation updates the equipment status to available, ensuring that the stored records remain accurate.

Exception handling has been included using Solidity require and revert statements to manage errors such as invalid equipment IDs, duplicate equipment entries, empty input values, invalid menu choices, and attempts to issue unavailable equipment. When a condition fails, the transaction is reverted with an appropriate error message.

Overall, the smart contract reduces the difficulties of manual equipment tracking and demonstrates the practical use of Solidity concepts such as structures, mappings, functions, condition checking, events, blockchain state, user-provided function inputs, and exception handling. The menu-driven function also provides a simple way to perform the main laboratory equipment operations through Remix IDE.